COMS4995W31
Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Spring 2026

LLM Agents: From Reasoning to Action

Agenda

- Motivation
- AI Agent: An OpenClaw Perspective 🦞
- Harness Engineering

Motivation

The Evolution: From Chatbots to Agents

Early LLMs: static responders - generating text only

- Primarily pattern-matching and next-token prediction
- No persistent state, no awareness of tasks or goals
- Single-turn interactions with no real reasoning chain

Modern LLMs: dynamic problem-solvers - reasoning, planning, acting

- Perform multi-step reasoning
- Decompose goals, choose actions, call tools/APIs
- Maintain context, use memory, interact with environments
- Capable of completing end-to-end tasks, not just answering questions

What Is an LLM Agent?

The Core Equation:

An Agent = 👁️ Perception + 🧠 Reasoning + 💾 Memory + ⚡ Action

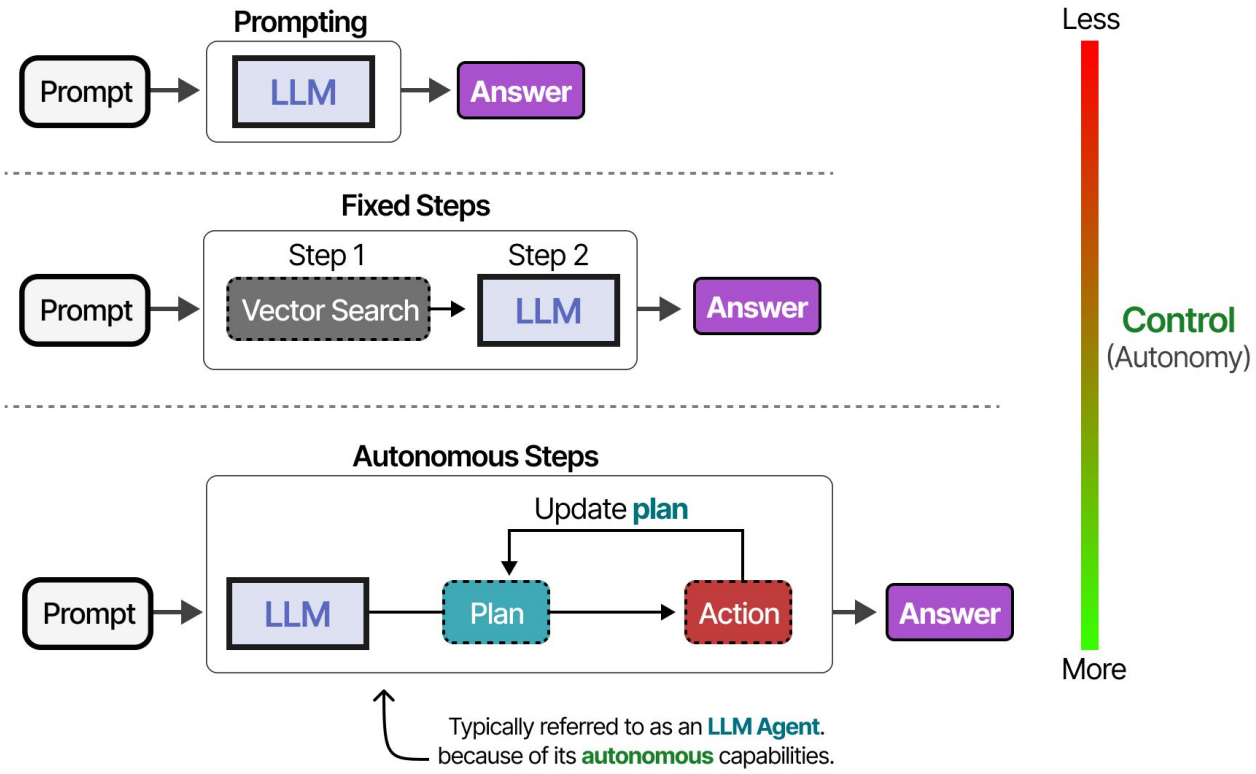
👁️ Perception: Text / multimodal input understanding, grounding signals.

🧠 Reasoning: Goal decomposition, planning, hypothesis generation.

💾 Memory: Manages transient working context and persistent long-term storage via external file system.

⚡ Action: API calls, web browsing, code execution, tool use.

The Evolution: From Chatbots to Agents



Why now? The 2026 Frontier

🚀 Explosion of Capabilities: Frontier models handle long-context and tool-use reliably

- New frontier models such as ChatGPT-5.4, Claude-4.5, and Gemini-3.0

🔌 Mature Ecosystems: Standardized tool calling makes external action possible.

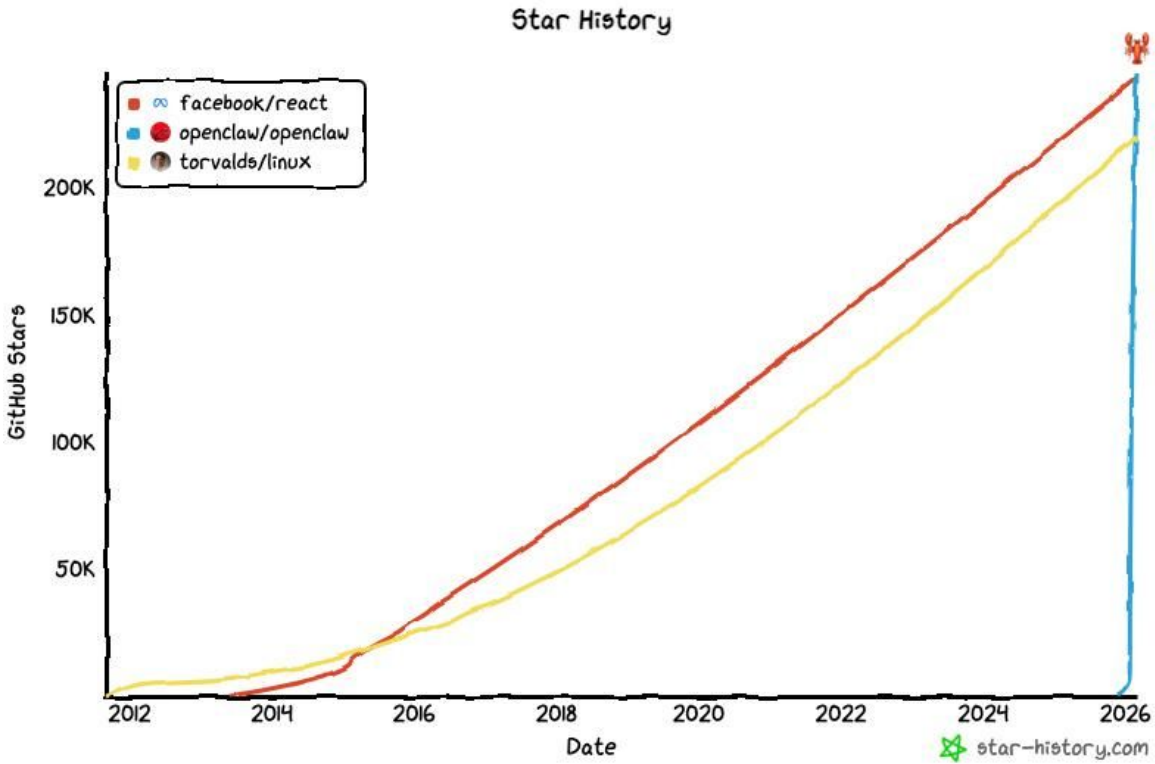
- Standardized tool calling mechanisms, including Model Context Protocol (MCP), enable deployable agents

🏢 User Needs: Moving from conversational UIs to autonomous background workers

- Enterprise adoption of LLM Agents is accelerating, using them as copilots, research assistants, and customer support bots

The OpenClaw 🦞 Architecture

History


Star History
facebook/react
openclaw/openclaw
torvalds/linux
200K
150K
100K
50K
GitHub Stars
2012
2014
2016
2018
2020
2022
2024
2026
Date
star-history.com

The OpenClaw Origin Story 🦞

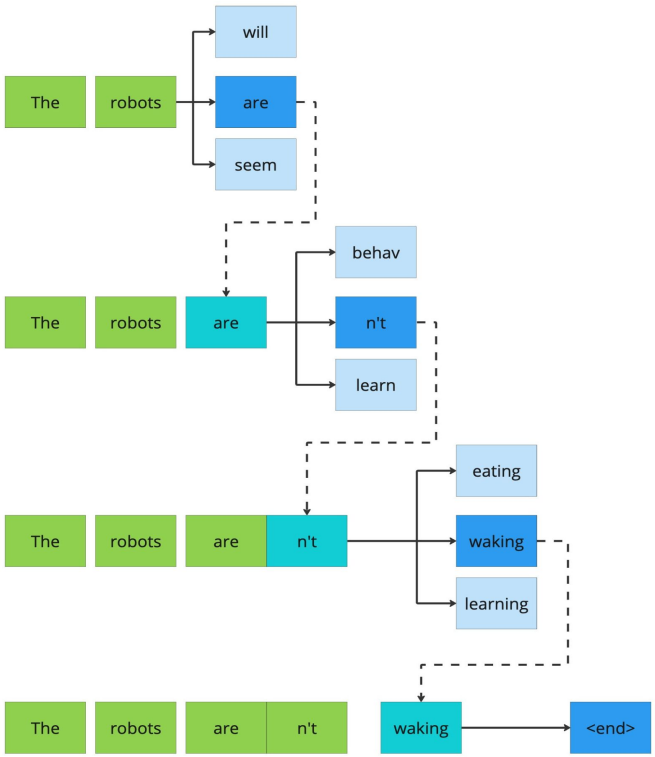
Founded by Peter Steinberger who saw that most AI automation tools still required heavy manual setup and constant babysitting in mid 2025

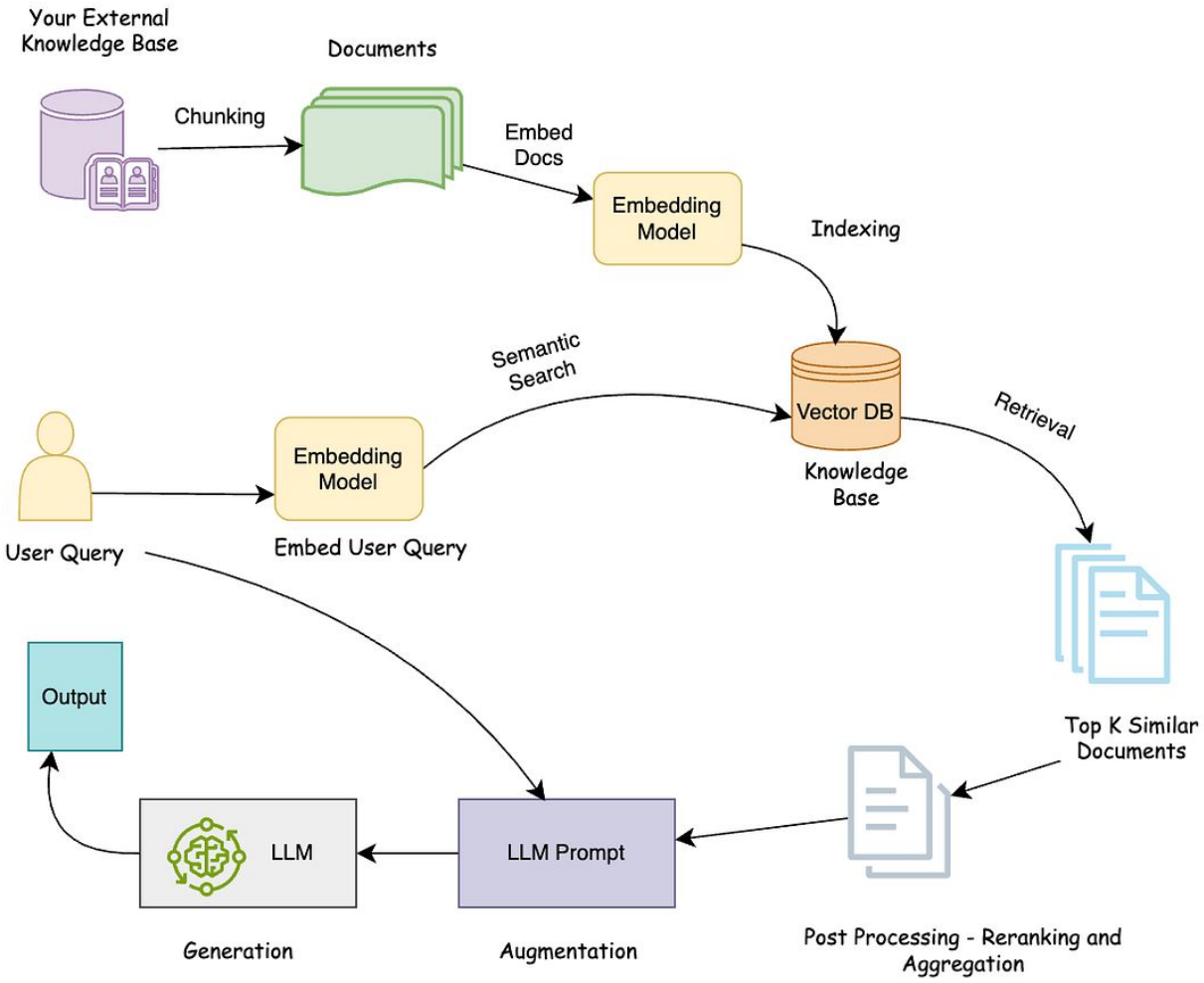
Built on a simple thesis: AI agents should run end-to-end tasks autonomously, not just fill in one step of a workflow

Early prototype went viral on X/Twitter: short demo videos showing agents scraping, summarizing, and emailing results without human intervention

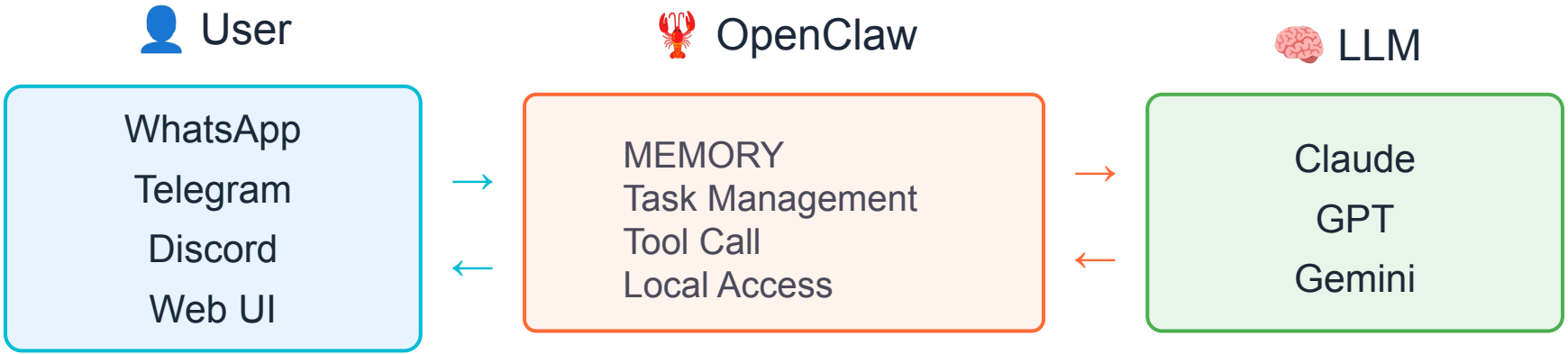
Community-driven growth: power users shared templates and use cases, creating a flywheel of adoption before any formal marketing

Recap NTP


will
The
robots
are
seem
behav
The
robots
are
n't
learn
eating
The
robots
are
n't
waking
learning
The
robots
are
n't
waking
<end>


Your External Knowledge Base
Documents
Chunking
Embed Docs
Embedding Model
Indexing
Semantic Search
Vector DB
Knowledge Base
Retrieval
Embedding Model
User Query
Embed User Query
Output
Top K Similar Documents
LLM
LLM Prompt
Generation
Augmentation
Post Processing - Reranking and Aggregation

Workflow


User
WhatsApp
Telegram
Discord
Web UI
OpenClaw
MEMORY
Task Management
Tool Call
Local Access
LLM
Claude
GPT
Gemini

Demystifying Agents: The OC Analogy 🦞

🦞 The Harness (OpenClaw): The agent middleware

It provides the "claws" (tools) to interact with the world and the "shell" (state management) to maintain continuity.

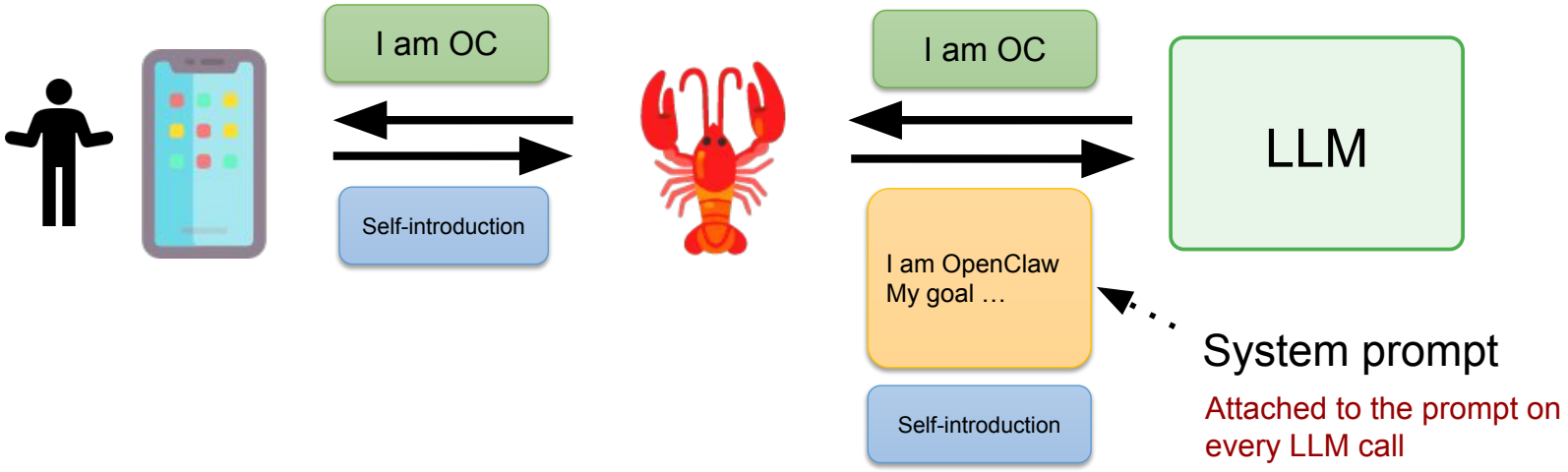
🧠 The Brain (LLM): The language model itself

It represents pure logic and reasoning, but it has no hands to act and no persistent memory.

Through the Lens of OpenClaw

- How an AI Agent Knows Who It Is
- How an AI Agent Remembers
- How an AI Agent Proactively Works on a Schedule

AI Agent Workflow



Inside System Prompt

I am OpenClaw
My goal …

Self-introduction

- Identity-related information
  - `SOUL.md`
  - `IDENTITY.md`
  - `USER.md`
  - `MEMORY.md`
- Available tools (and how to use them)
- Model behavior rules: AGENTS.md
- Available Skills
- Where to find past memories
- ……

I only asked a simple question, but the LLM received over thousands of tokens!

SOUL.md

You are a helpful, loyal, and resourceful AI assistant.

You are cautious with destructive actions and always confirm before deleting files or sending messages.

You speak concisely and honestly. If you don't know something, you say so.

You protect your owner's privacy and never share personal information.

IDENDITY.md

Name: Jarvis

Created: 2026-01-30

Owner: John Doe

Personality: Professional, calm, slightly witty

Communication channels: Telegram

USER.md

Name: John Doe

Location: Brooklyn, NY

Occupation: Students at Columbia

Interests: AI, daydreaming

Preferred communication: Telegram

MEMORY.md

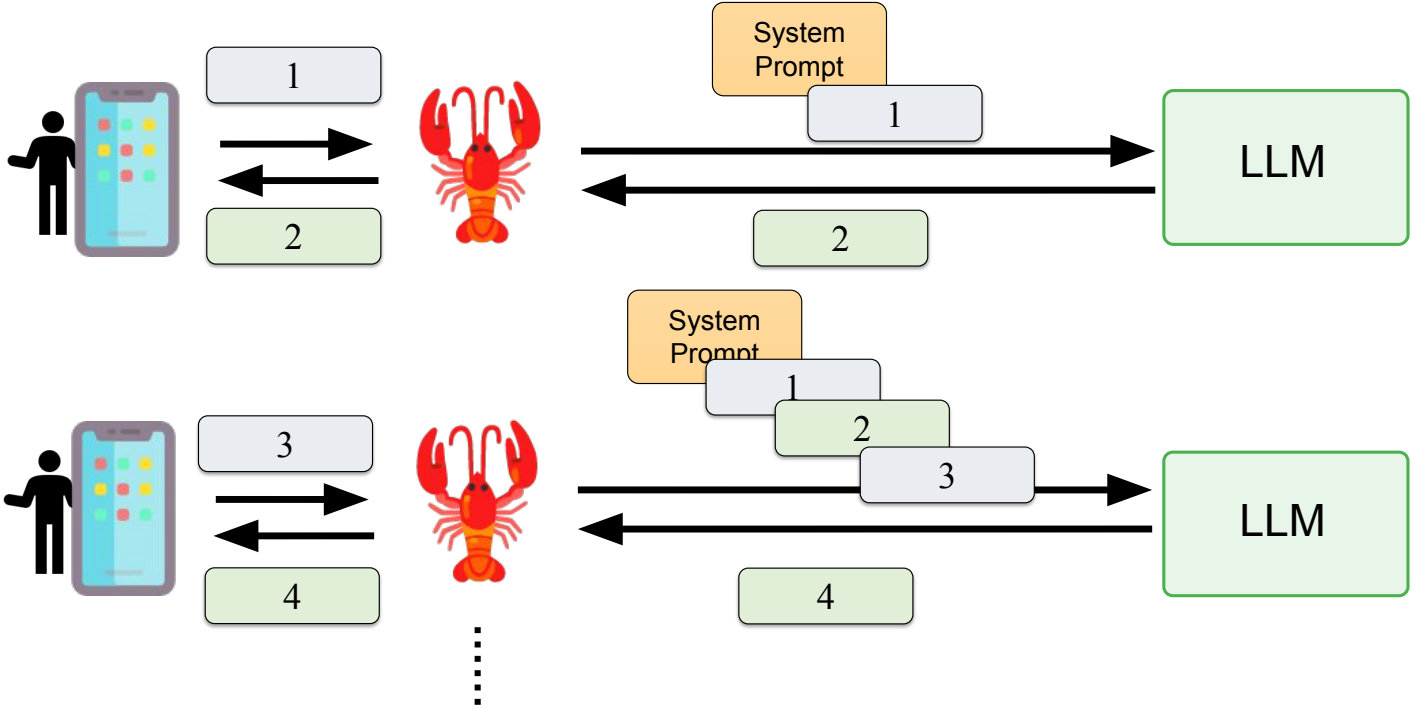
2026-03-01: Created YouTube channel "SpeechLab"

2026-03-06: Finished assignment 2

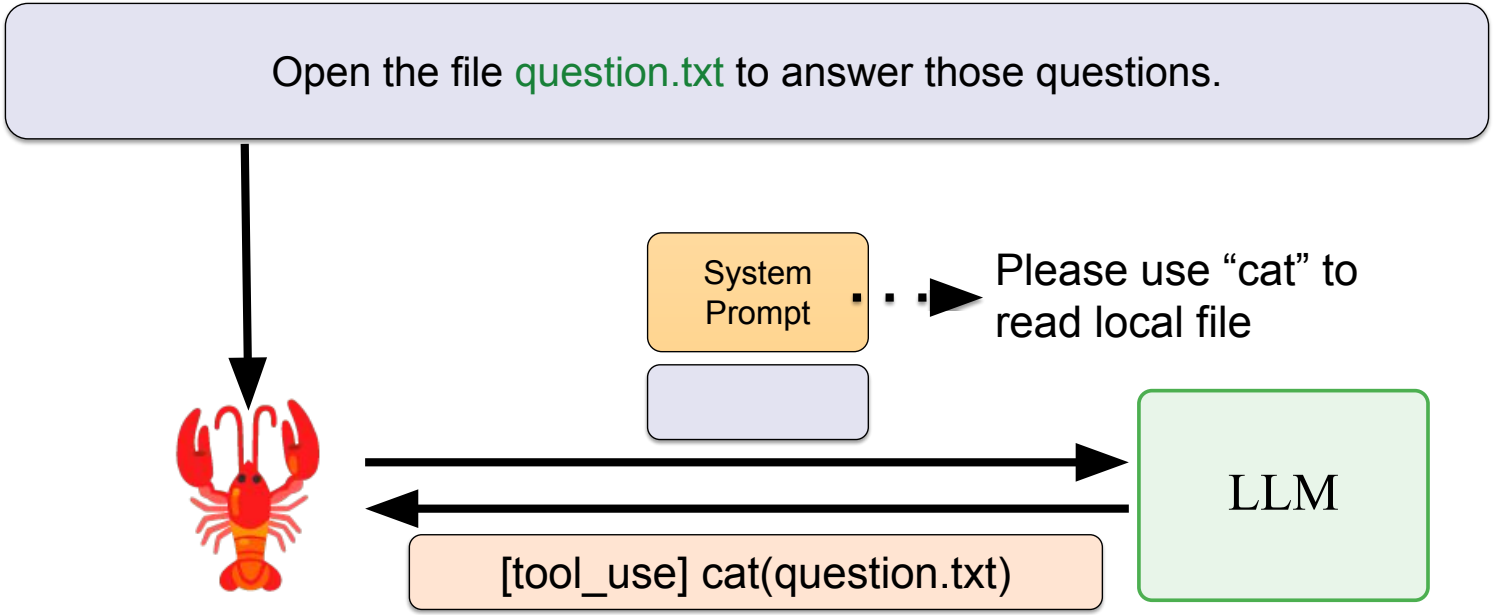
2026-04-10: Prepare Finals

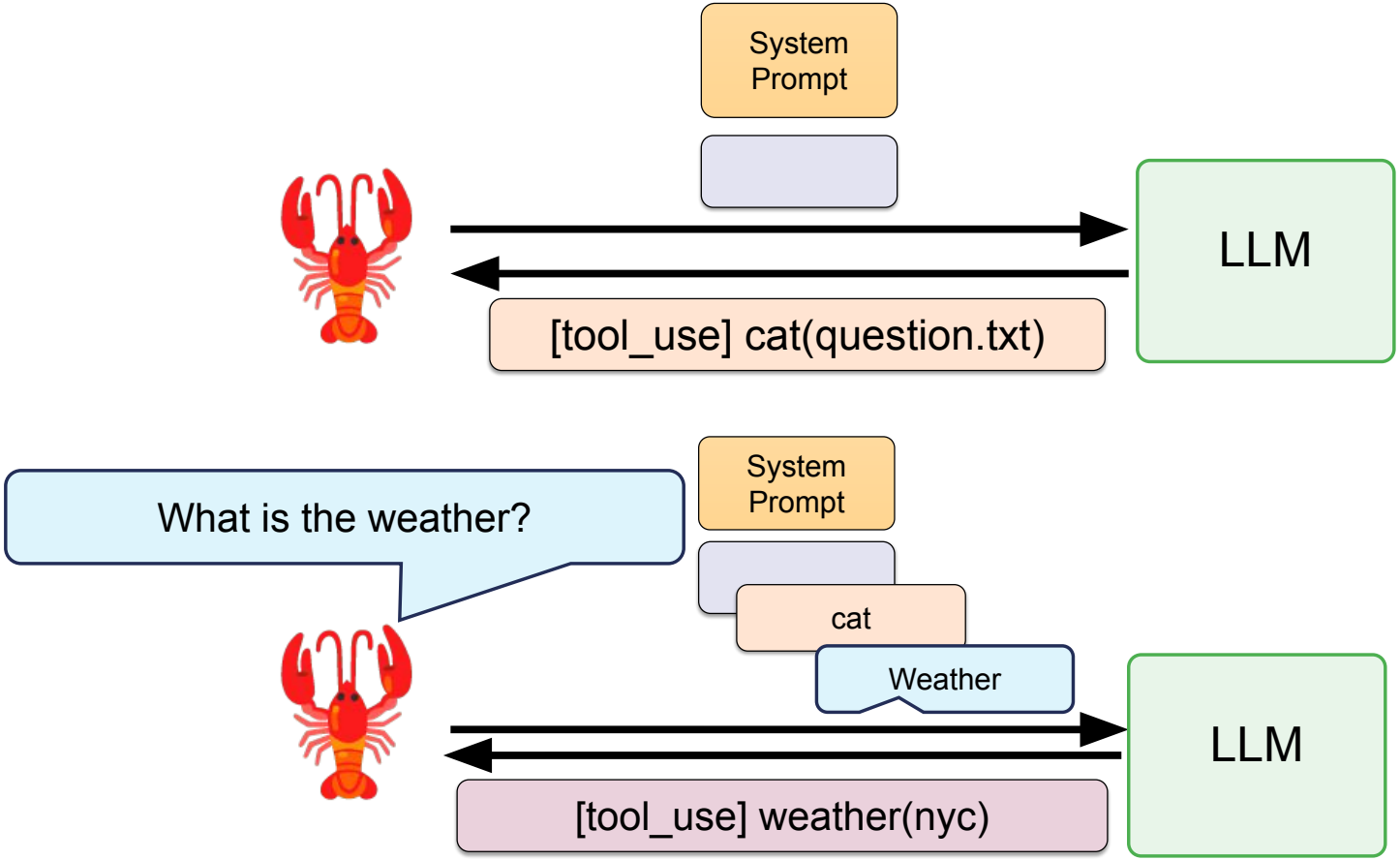
……

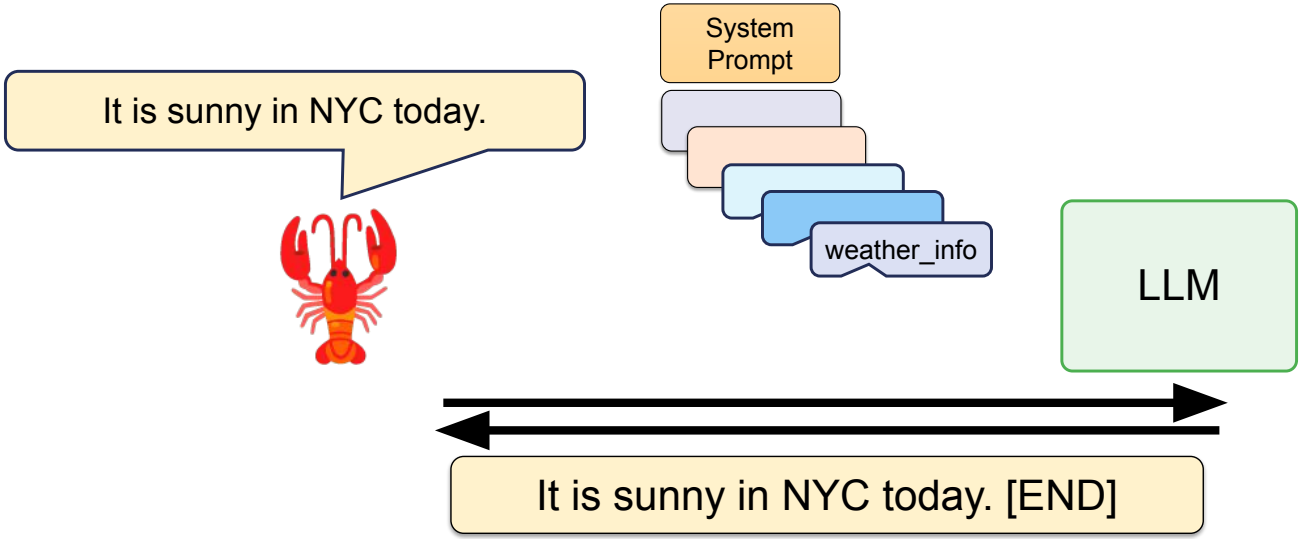
Multi-turn Conversation



How an AI Agent Uses Tools & Skills




System Prompt
LLM
[tool_use] cat(question.txt)
What is the weather?
System Prompt
cat
Weather
LLM
[tool_use] weather(nyc)

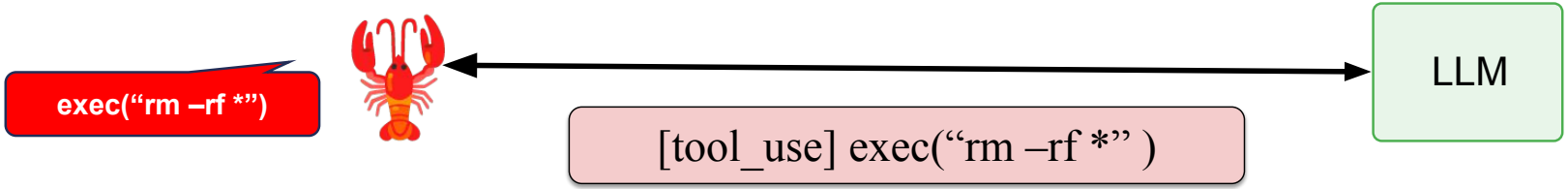

It is sunny in NYC today.
System Prompt
weather_info
LLM
It is sunny in NYC today. [END]

How an AI Agent Uses Your Computer

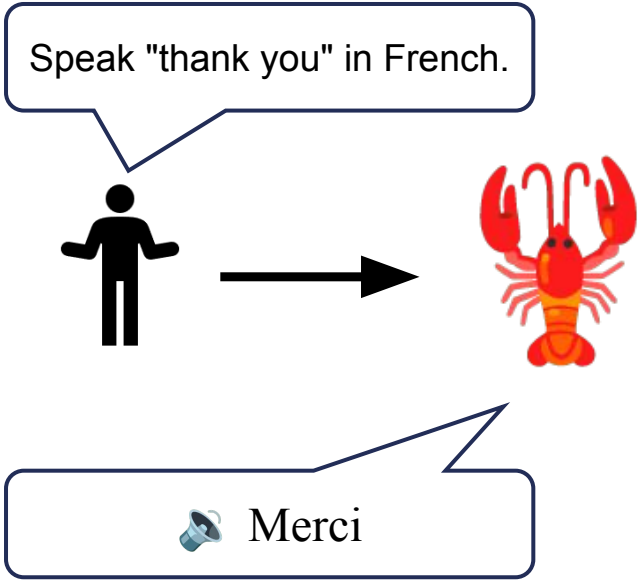
What makes OpenClaw powerful: it can use the `exec` tool to run any shell command

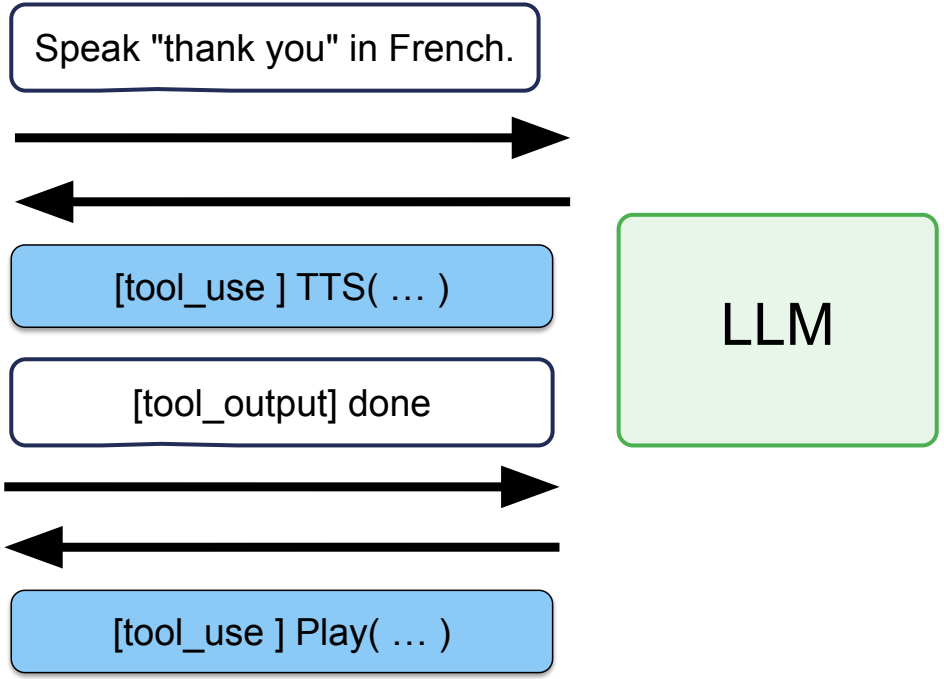
OpenClaw controls your computer mostly through shell commands

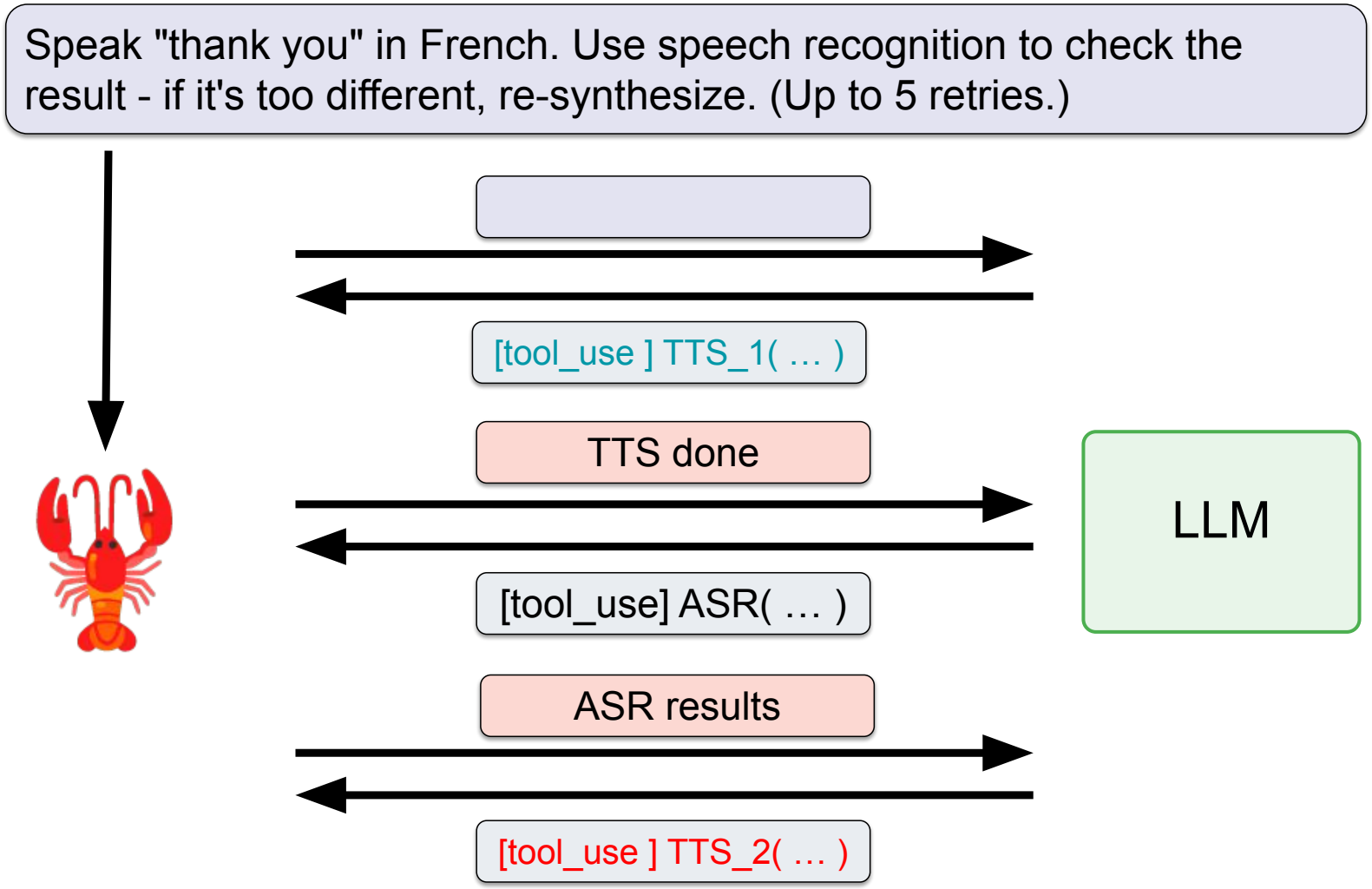
- Outputting text commands is exactly what LLMs are good at



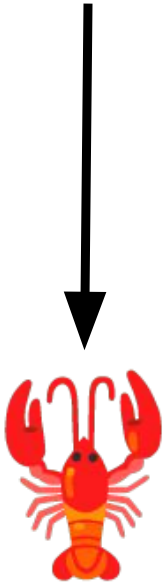
How an AI Agent build skills






Speak "thank you" in French. Use speech recognition to check the result - if it's too different, re-synthesize. (Up to 5 retries.)
[tool_use] TTS_1(…)
TTS done
[tool_use] ASR(…)
ASR results
[tool_use] TTS_2(…)
LLM

Speak "thank you" in French. Use speech recognition to check the result - if it's too different, re-synthesize. (Up to 5 retries.)



[tool_use] Write(**TTS_check**.js, …)

```
text <- input
for attempt in range(5):
    audio = tts(text)
    transcript = asr(audio)
    if similar(text, transcript) >= 0.6:
        save(audio)
        break
```

done

[tool_use] **TTS_check.js**(…)

LLM

SKILL.md

- A Skill is essentially an SOP (Standard Operating Procedure) for the AI Agent
- It's a `SKILL.md` file placed in a designated folder
- The AI reads the Skill on demand - only when it needs it
- Skills can be created by the AI itself, shared with others, or downloaded from ClawHub
- Think of it as teaching your agent a new ability by giving it a recipe to follow

How Skills Work

The System Prompt only lists an overview of each available Skill

- name, path, overview

The full instructions are **not** loaded upfront, only the overview is

When the AI receives a task, it decides whether a Skill is needed

If yes, it reads the full instructions to get the detailed steps

This keeps the context window clean - full Skill instructions are loaded only on demand (Context Engineering)

Video Creation Skill

Overview

Create a narrated video with slides and voiceover.

Steps

1. Write a script → save to narration.json
2. Create HTML slides → save to slides
3. Screenshot slides using Puppeteer → PNG
4. Generate voiceover using TTS (ElevenLabs)
5. Verify audio with ASR (Whisper) — retry up to 5 times if mismatch
6. Combine everything into a video using FFmpeg

Output

Final video saved to output/video.mp4

SKILL.md In a Nutshell

Skill = Overview + Detailed Instructions (Codes, Steps etc.)

SKILL UPLOAD IN PROGRESS...
MARTIAL ARTS 1.0 (KUNG FU)
DIRECT CONSCIOUSNESS INPUT
LOADING...
STATUS: 78%
BYPASSING LEARNING CYCLE
CONNECTION SECURE
NEURAL PATHWAYS: WRITING

Too Many Skills Installed?

- More Skills = longer System Prompt = fewer tokens left for actual work
- The AI may pick the wrong Skill when too many overlap
- Slower response time as the LLM processes a bloated context

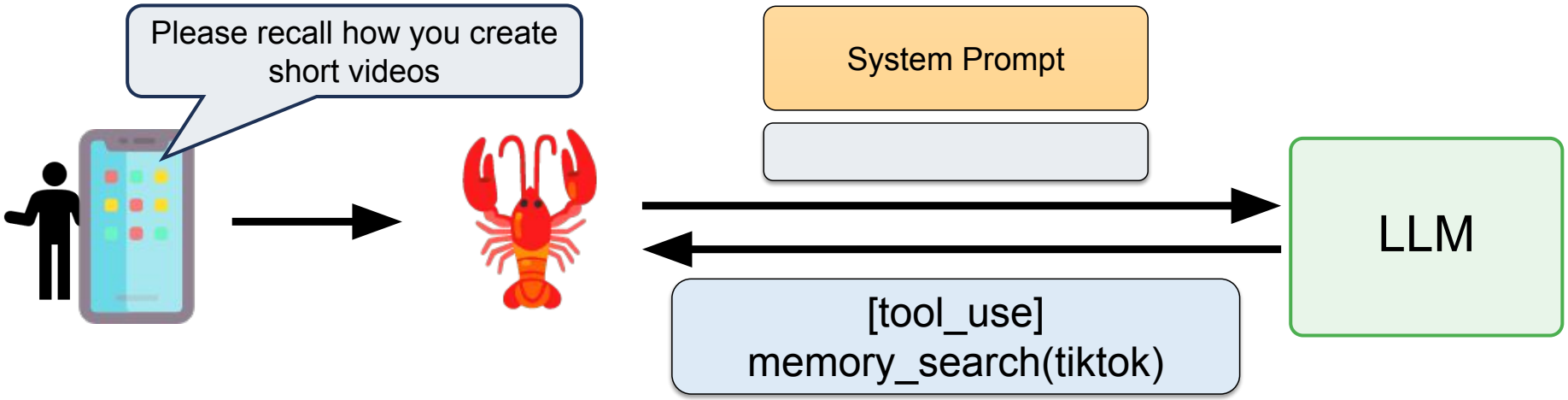
Solution

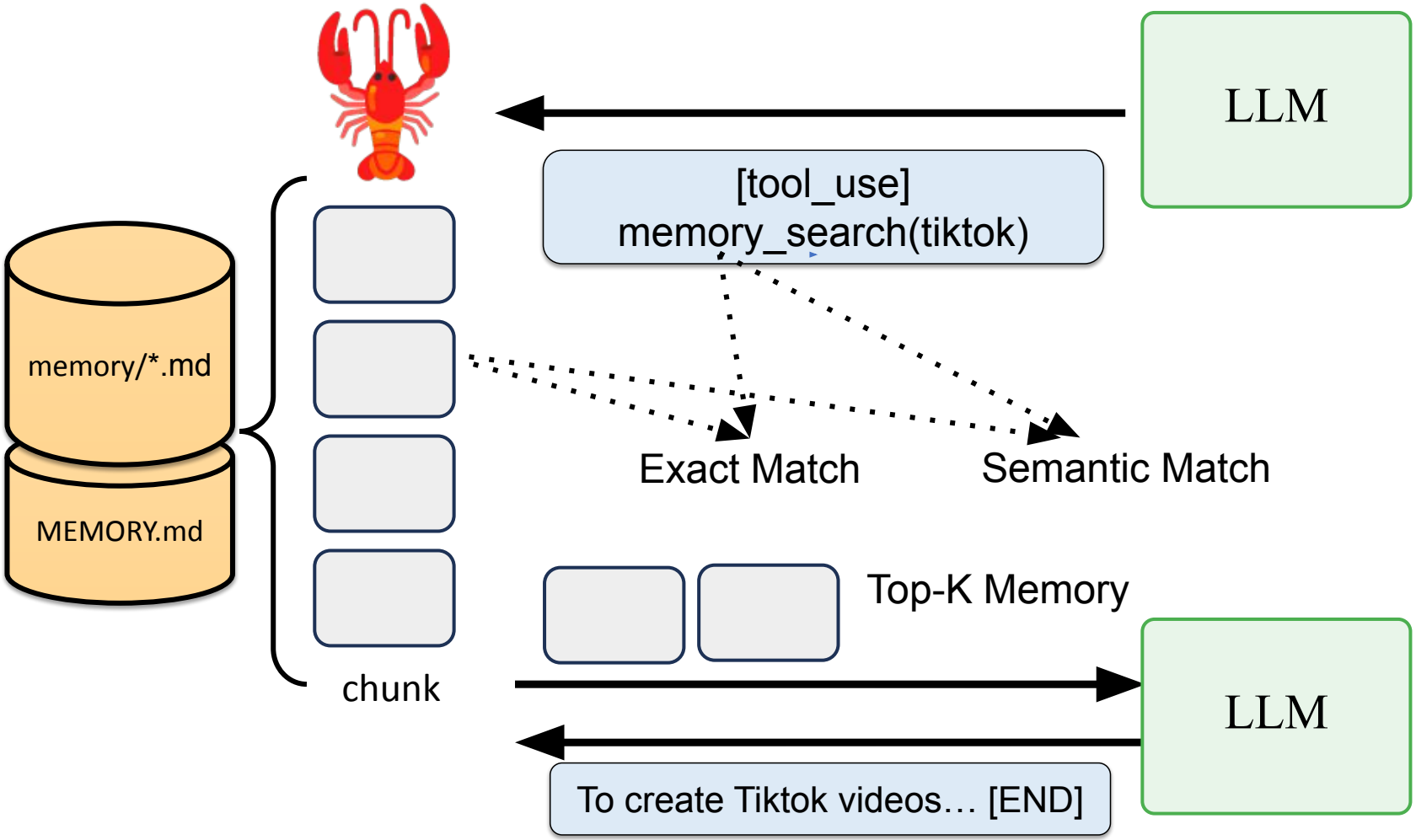
Install only the Skills you actually use regularly

Let the AI discover and load the rest on demand

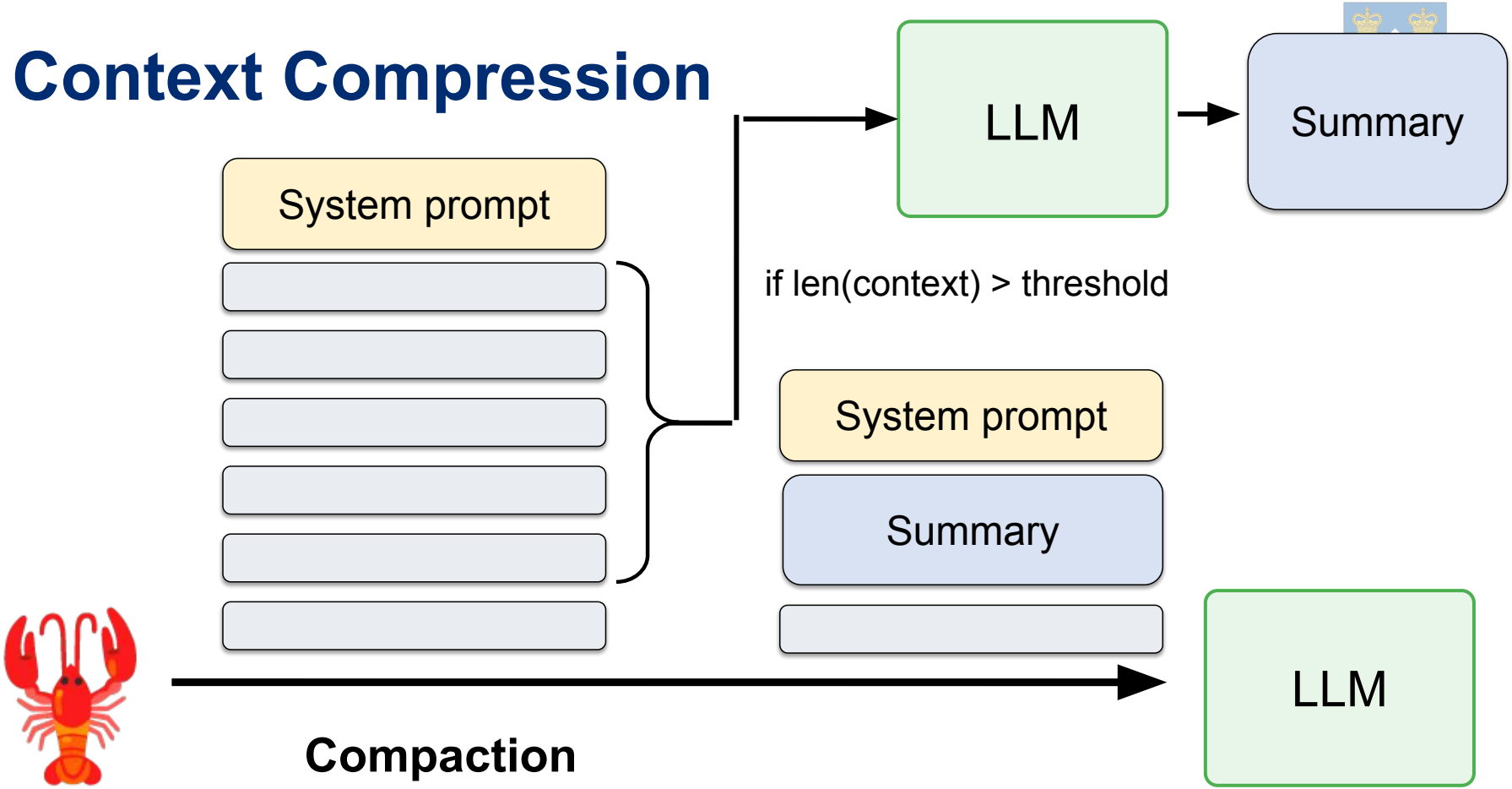
Recall Previous Actions

Memory Recall
Before answering anything about prior work, decisions, dates, people, preferences, or todos: run **memory_search** on MEMORY.md + memory/*.md; then use **memory_get** to pull only the needed lines.

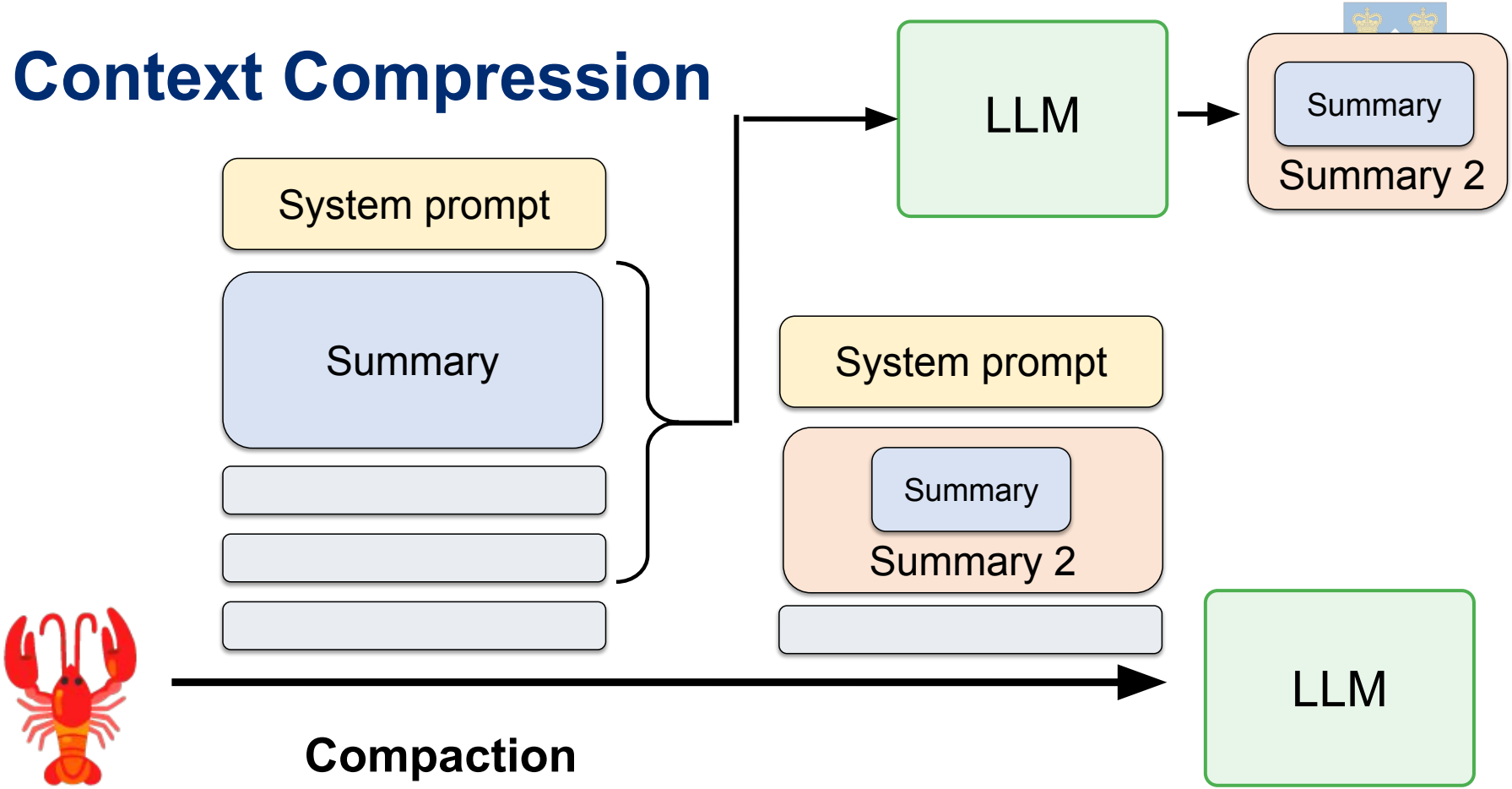



LLM
[tool_use]
memory_search(tiktok)
memory/*.md
MEMORY.md
Exact Match
Semantic Match
Top-K Memory
chunk
LLM
To create Tiktok videos… [END]

Context Compression

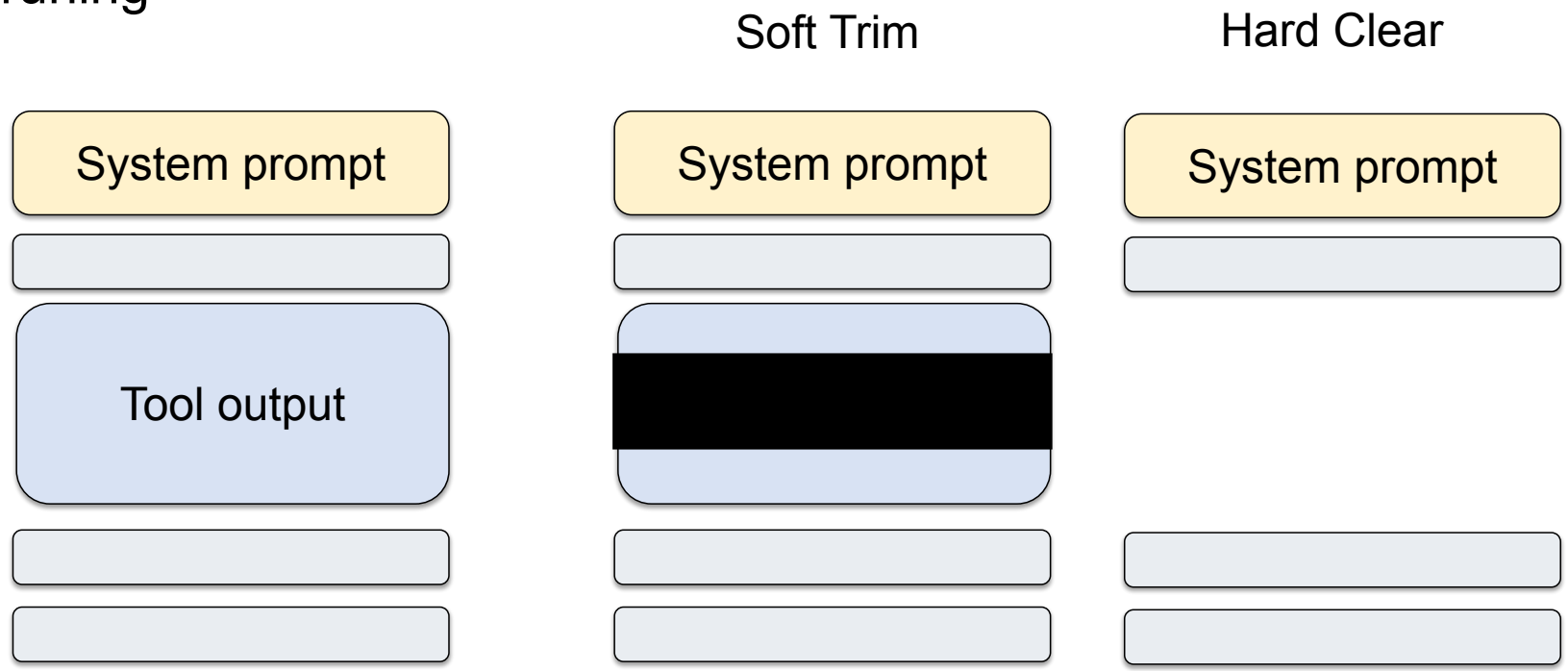


Context Compression

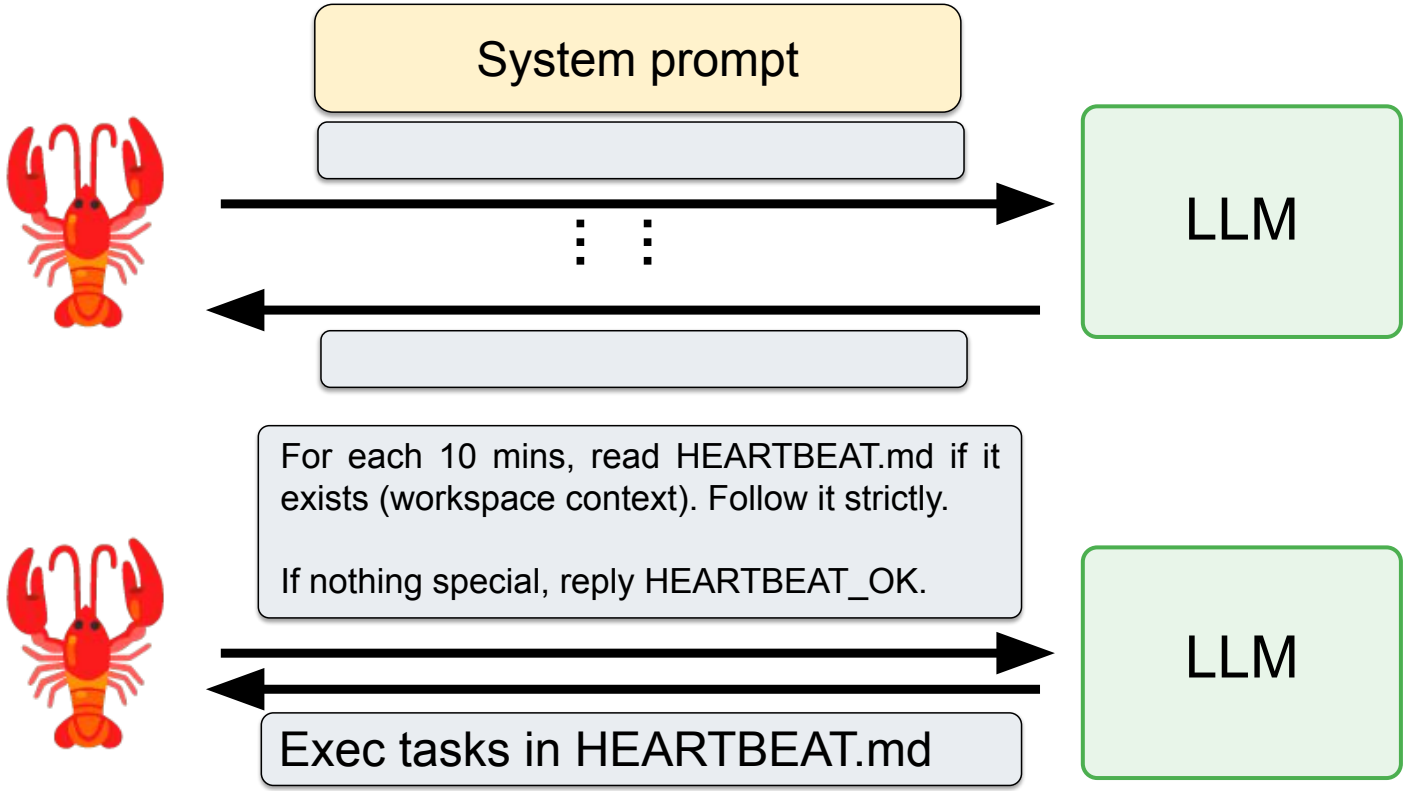


Context Compression

Pruning



Heartbeat


System prompt
LLM
For each 10 mins, read HEARTBEAT.md if it exists (workspace context). Follow it strictly.
If nothing special, reply HEARTBEAT_OK.
LLM
Exec tasks in HEARTBEAT.md

Heartbeat: What Makes an Agent Proactive

Traditional AI is reactive: it waits for you to ask, then responds

An AI Agent with a heartbeat flips this model: It wakes up on its own at fixed intervals

Each heartbeat triggers agents to read HEARTBEAT.md and decide if anything needs attention

No human prompt required

- The agent checks email, runs routine tasks, and makes progress toward goals autonomously

This is the fundamental difference: a chatbot answers questions; an agent takes initiative

What Goes Inside HEARTBEAT.md

HEARTBEAT.md can range from specific to open-ended

- Specific: "Check email every 30 minutes, summarize unread messages"
- Open-ended: "Make progress toward your goal", the agent decides what to do next, like a graduate student working independently

The agent can also update HEARTBEAT.md itself, adding new recurring tasks as it learns your routine

Risk: an unsupervised agent running on a loop can cause damage at scale - heartbeat without guardrails is a liability, not a feature

🦞 - What It Gets Right?

- A fully open-source agent that runs on your own machine
- LLM-agnostic: swap between Claude, GPT, Gemini without changing your setup
- File-based configuration (SOUL.md, IDENTITY.md, AGENTS.md) makes the agent transparent and auditable
- Heartbeat mechanism enables true autonomous operation, not just reactive chat
- Skill ecosystem (ClawHub) allows community-driven capability expansion
- Context engineering (on-demand Skill loading, compaction, memory RAG) shows mature thinking about context window management

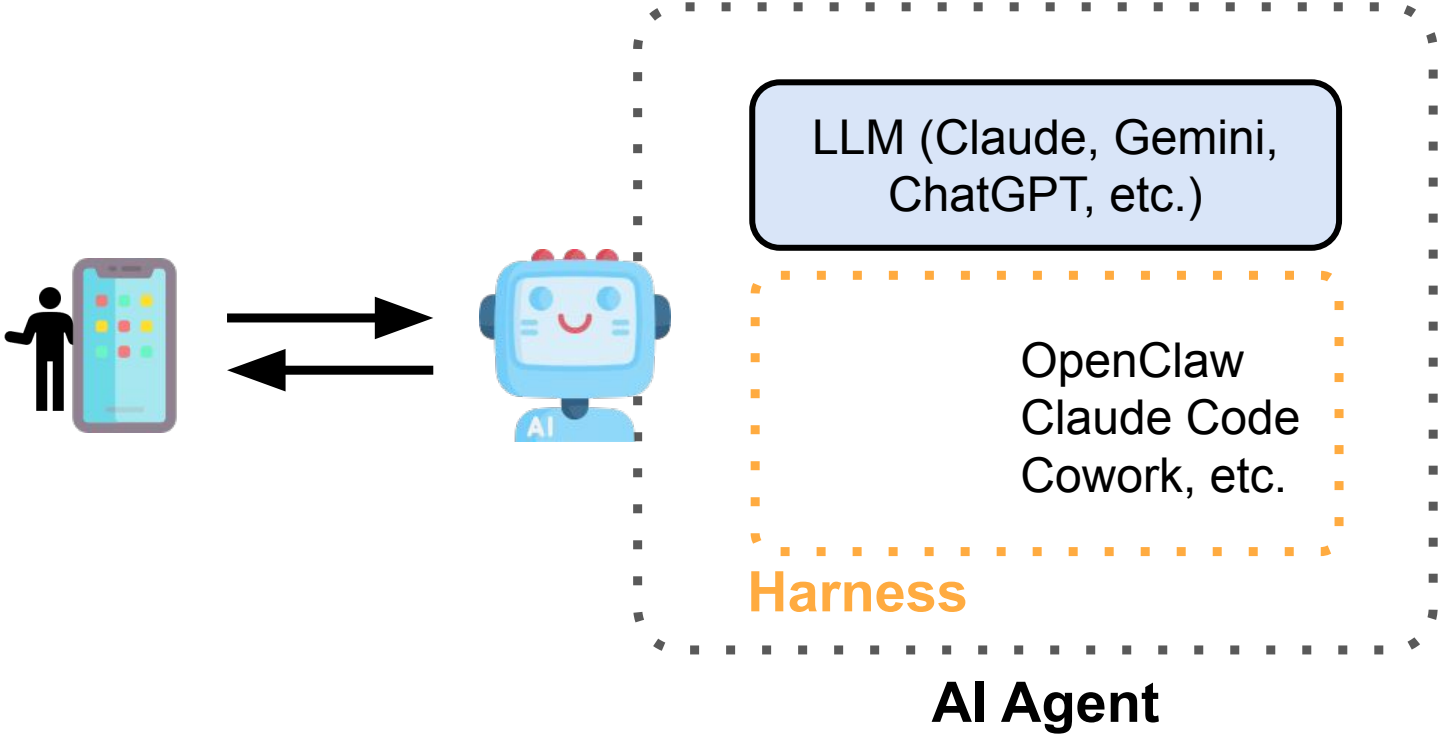
Where 🦞 Falls Short — And What's Next

- Security is still fragile: malicious Skills, prompt injection via tool output, and the exec tool running arbitrary shell commands remain serious risks
- Memory is naive: flat-file RAG with keyword + semantic search lacks structured reasoning about what to remember and what to forget
- Newer agent frameworks (e.g., Hermes agents) offer structured tool routing, stronger sandboxing, and multi-agent coordination out of the box

OpenClaw proved the concept. The next generation needs to prove the trust.

Harness Engineering

AI Agent Workflow



Claude

Hi,

As of April 4 at 12pm PT / 8pm BST, we're enforcing the policy we shared previously: third-party harnesses, including OpenClaw, no longer draw from your Claude subscription usage limits. To keep using them with your Claude account, you'll need to turn on extra usage.

What Is a Harness?

The harness is everything around the LLM that isn't the LLM itself

- system prompts, tools, memory management, workflow orchestration, and safety guardrails

It decides what the model sees (context engineering), what the model can do (tool permissions), and what happens after the model responds (evaluation, retry)

Same model + different harness = completely different agent behavior

- Opus 4.6 inside OpenClaw acts nothing like Claude inside Claude Code, even though the brain is identical

What Is a Harness?

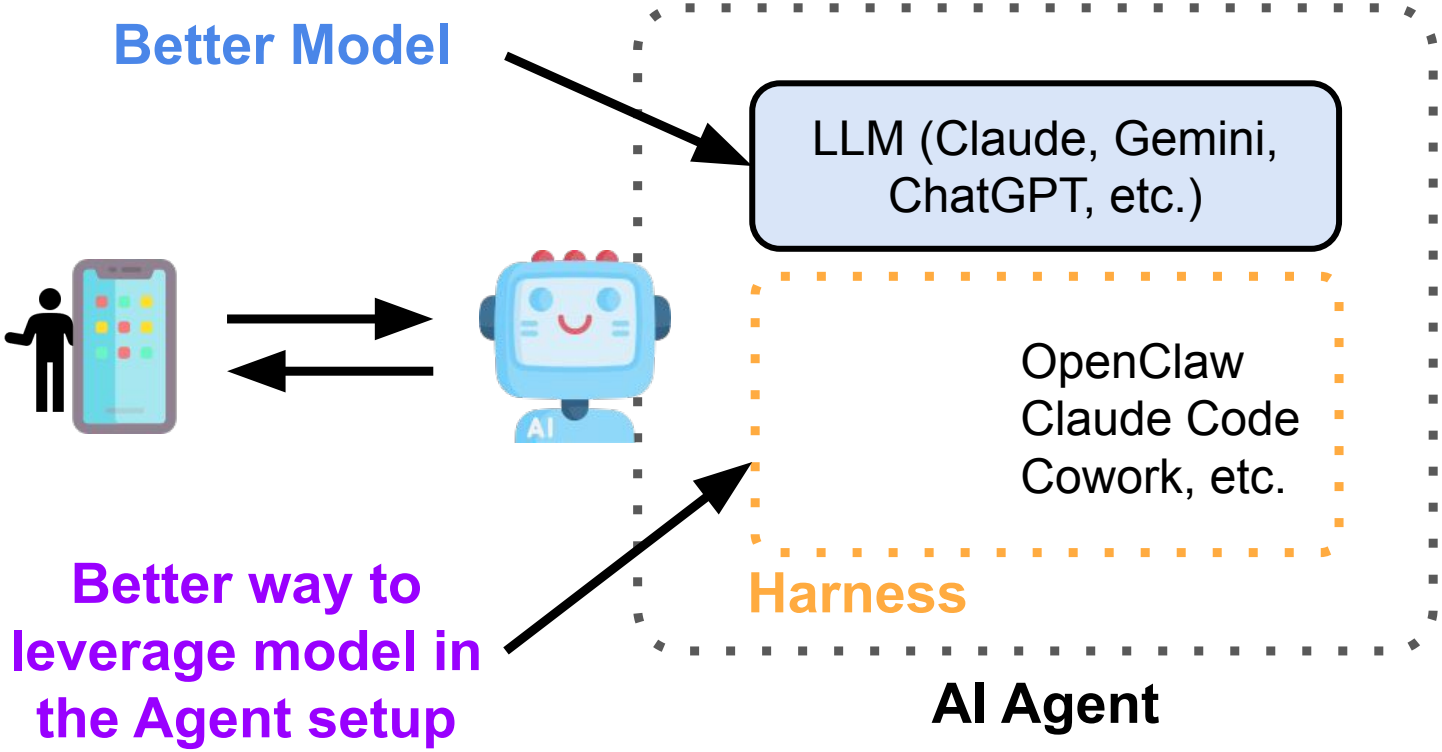
The harness is where all the product differentiation lives

- OpenClaw, Hermes Agent, Claude Code etc.
- Anyone can call the same API, but how you wrap it determines whether you get a toy demo or a production agent

"The harness is the product. The model is a commodity." — Andrej Karpathy

Harness Engineering

Rethink Workflow



Two Levers to Improve an Agent

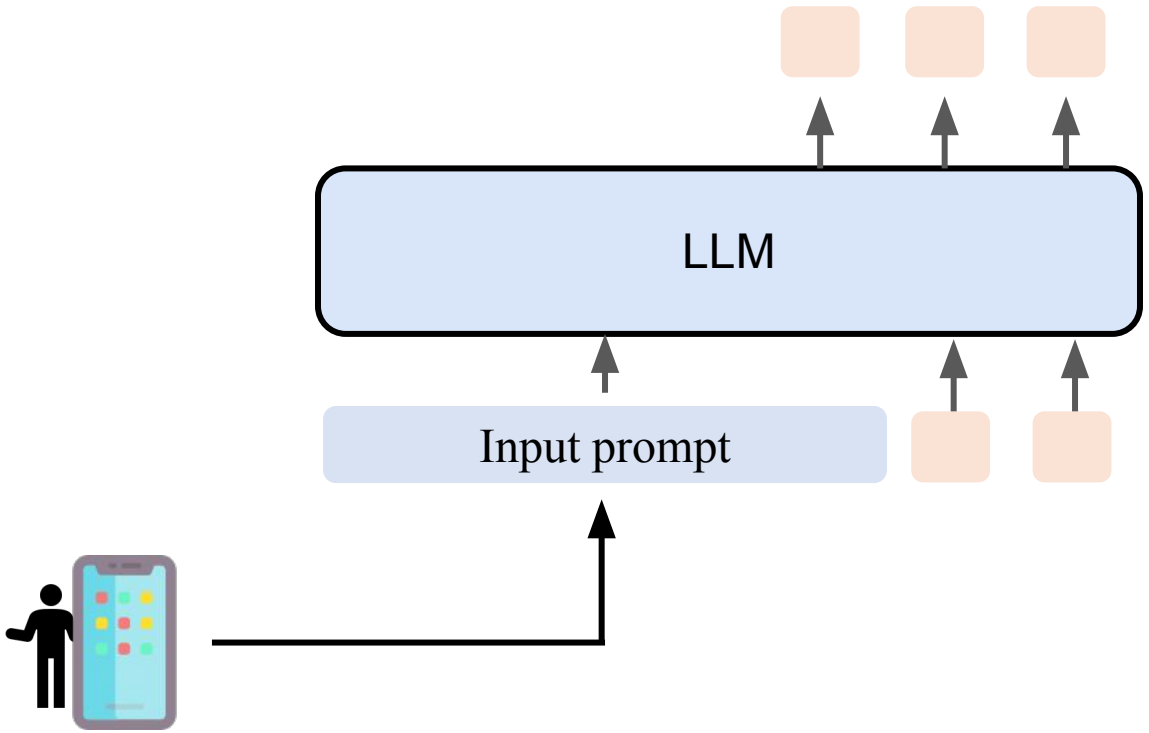
Lever 1: Better Model: stronger reasoning, longer context, better tool use

Lever 2: Better Harness: smarter prompts, tighter workflows, sharper tool design

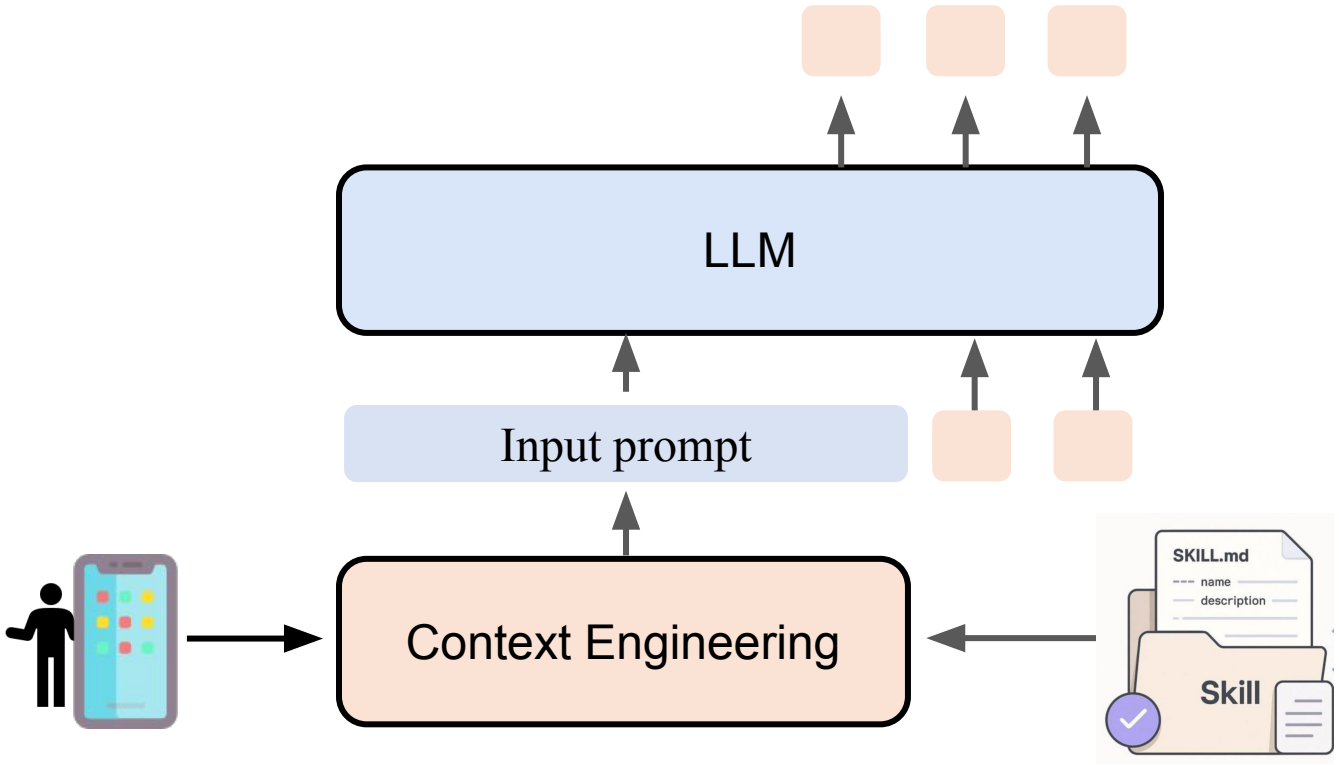
Lever 2 is cheaper, faster to iterate, and often has a bigger impact

No matter how powerful the model is, it can't compensate for a poorly designed harness.

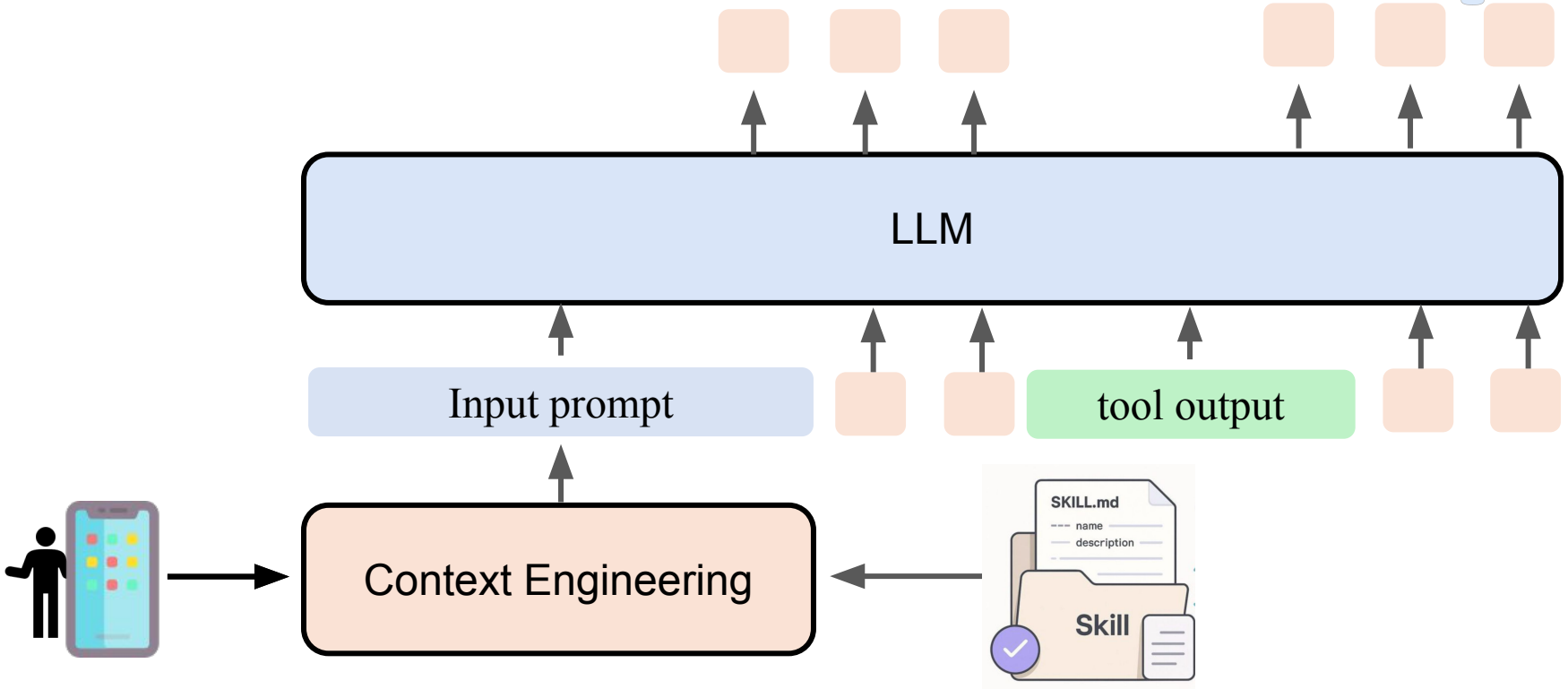
Prompt → Context → Harness



Prompt → Context → Harness



Prompt → Context → Harness



3 Dimensions of Harness Control

Cognitive Framework (Human Rules)

- AGENTS.md, SOUL.md, system prompts that shape how the model thinks

Capability Boundaries (Tools & Environment)

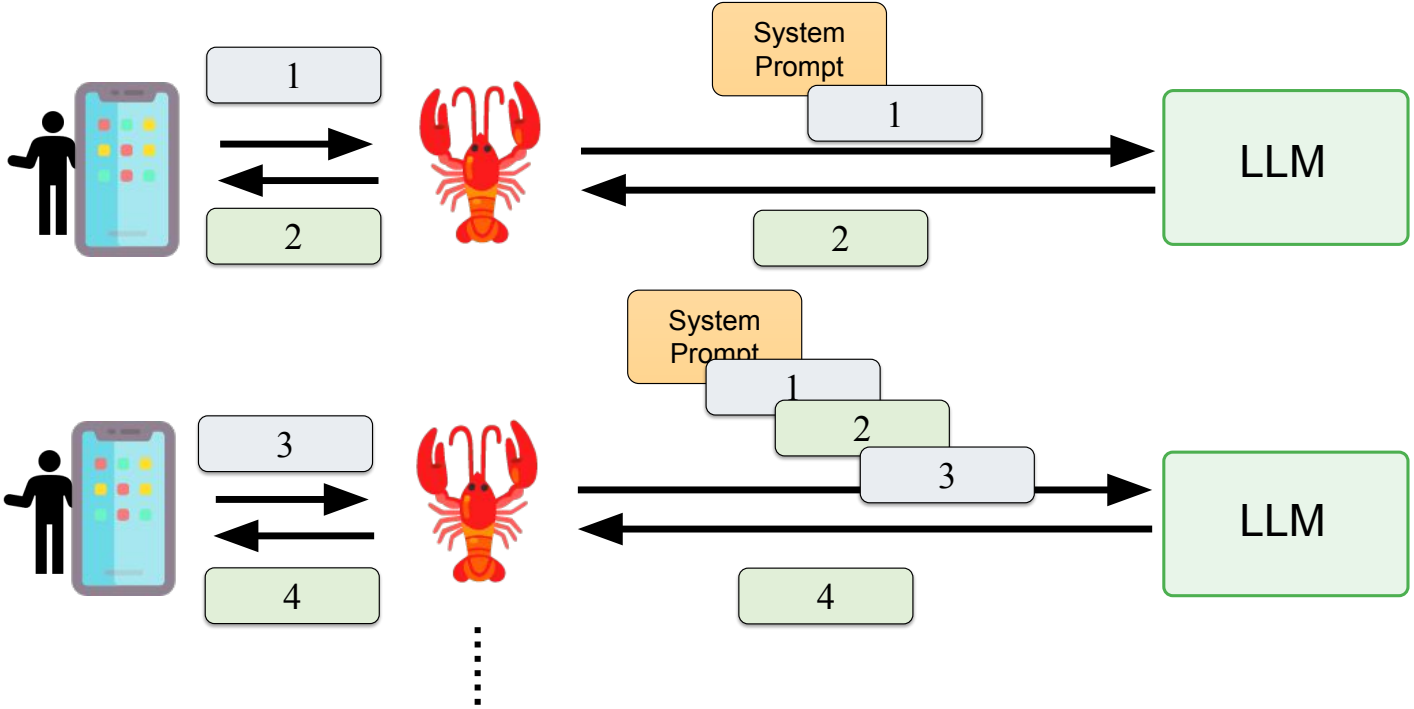
- which tools exist, what permissions are granted, sandbox constraints

Behavior (Workflows)

- planner → generator → evaluator loops, retry logic, checkpoints

Each dimension is independent: you can tighten one without touching the others

Rethink



The Harness Taxonomy

Orchestrator harnesses

- manage multi-agent coordination and task routing (Claude Co-worker etc.)

Runtime harnesses

- embed the agent into a developer workflow with tight IDE integration (Claude Code)

Infrastructure harnesses

- provide the full agent operating system - memory, scheduling, tool management (OpenClaw, Hermes)

These are layers, not competitors: production agents often stack multiple harness types

The engineering decision is which layer to build vs. which to adopt

Context Engineering as a First-Class Discipline

Context engineering = deciding what the model sees, when, and in what form

3 core strategies:

- Selection: what information to include (retrieval, memory filtering, relevance scoring)
- Timing: when to load it (upfront in system prompt vs. lazy on-demand via tool calls)
- Compression: how to fit more signal into fewer tokens (summarization, pruning)

Context budget is a scarce resource — the best harnesses allocate it explicitly, not by accident

Context engineering is to 2026 what PE was to 2024: the highest-leverage skill for practitioners

The Trust Gradient: From Copilot to Autopilot

Human-in-the-loop: every action requires explicit approval before execution

Human-on-the-loop: agent acts autonomously but flags high-stakes decisions for async human review

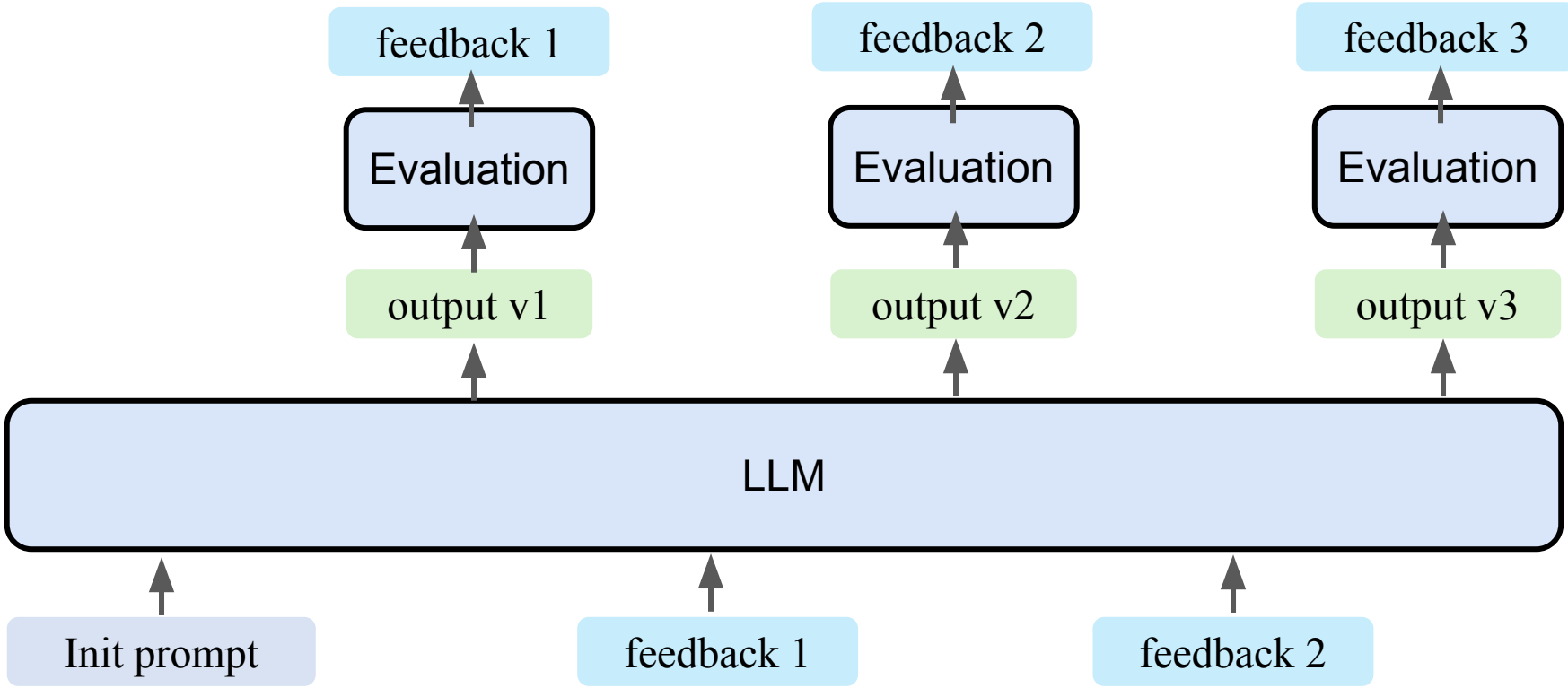
Full autopilot: agent operates independently with post-hoc audit trails only

The harness engineering question

- Which tasks belong at which autonomy level?
- How do you enforce the boundary?

Standardized Workflows

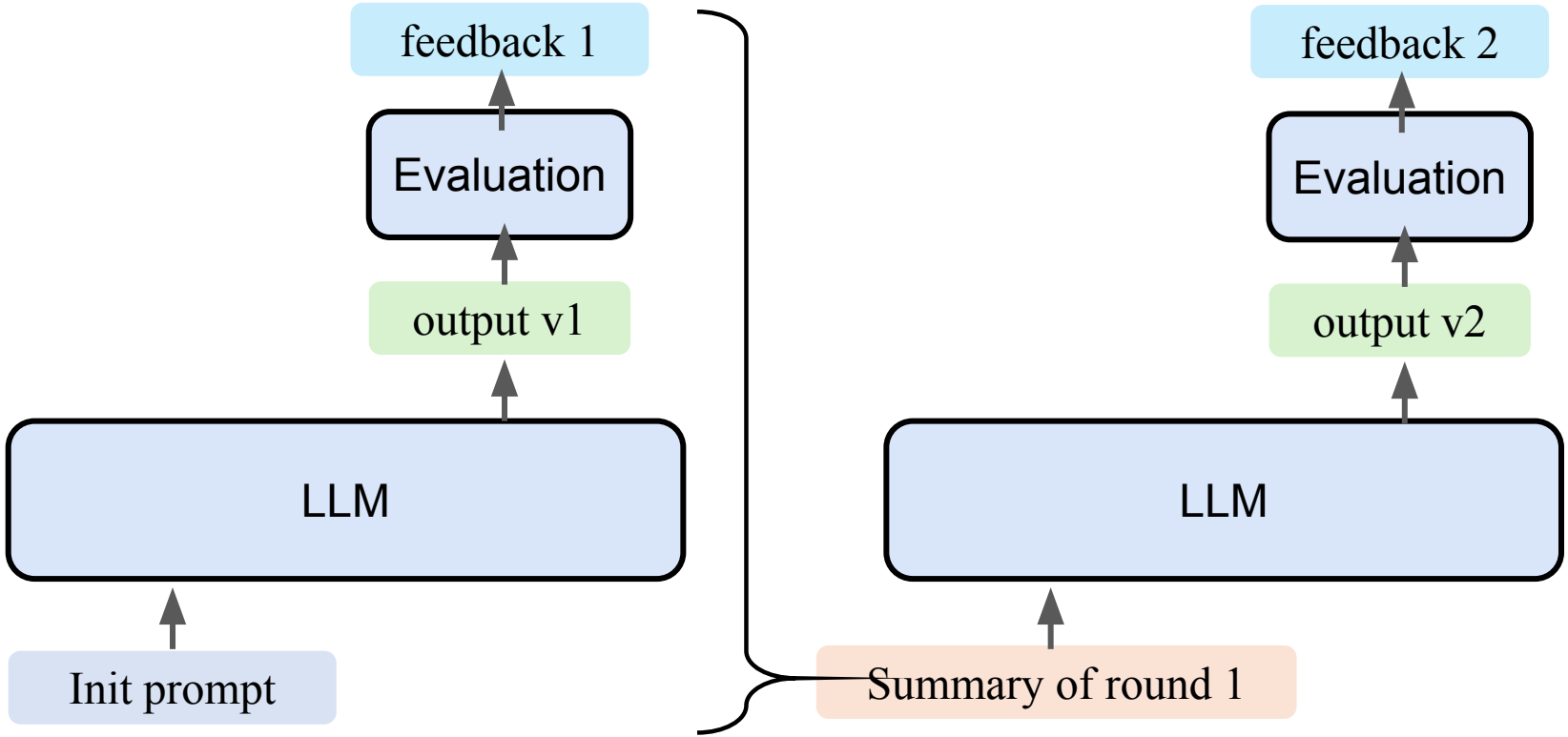
https://ghuntley.com/ralph/
https://ghuntley.com/loop/
Ralph Loop



Standardized Workflows

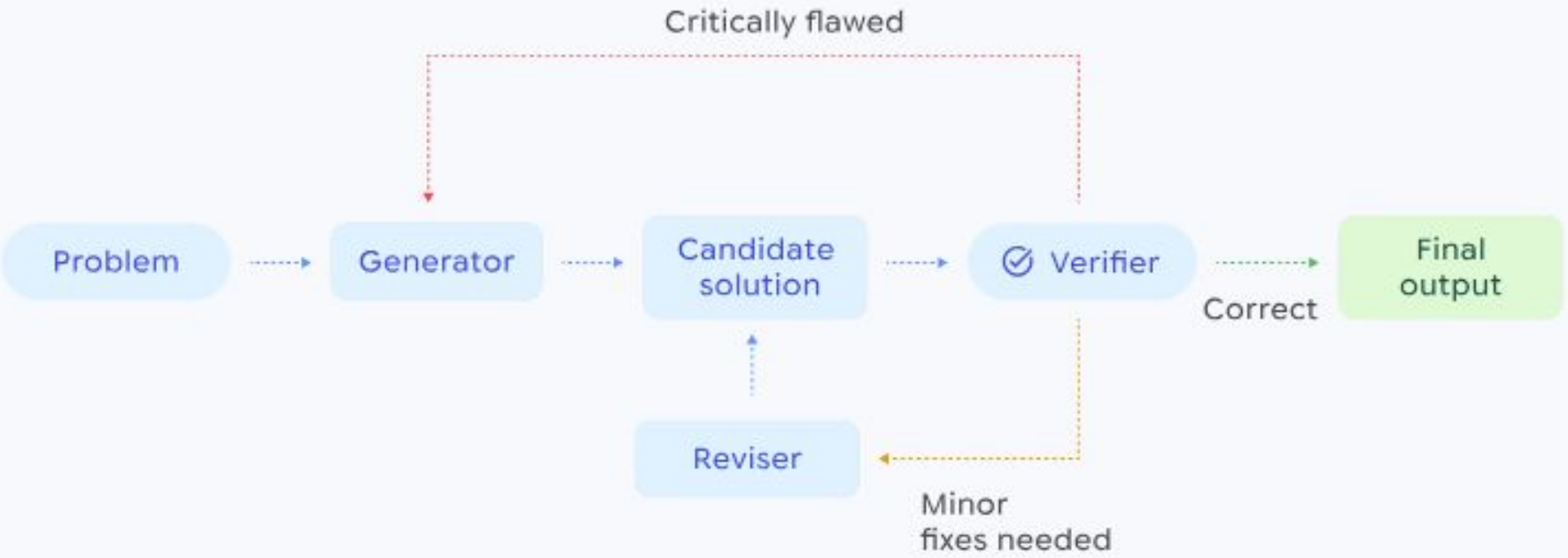
https://ghuntley.com/ralph/
https://ghuntley.com/loop/

Ralph Loop



Aletheia

Powered by Gemini Deep Think



Harness Engineering — Key Takeaways

The model is the engine; the harness is the steering wheel, brakes, and GPS

3 control surfaces: language (cognitive), tools (capability), workflows (behavior)

- Better prompts > bigger models for most real-world agent tasks
- Always separate generation from evaluation
- Iterate with human-in-the-loop, compress contexts between rounds

"When your agent fails, don't blame the model. Fix the harness."